Data Setup Automation Design

VMM PO Governance — Data Setup Automation: Design

Status: proposal for review · **Date:** 27 Oct 2026 · **Applies to:** POGovApi / Admin portal (this repository)

Decision asked for: stop uploading Excel / CSV files in 📂 Data Setup. The tool **fetches every dataset directly from its original source system** on a schedule — through a **SQL Server linked server** (a read-only database connection to the source) or through the **source's API** (SAP OData / REST) — checks that what it received is complete, and only then makes it the data everyone works with. If a pull fails, the previous data stays in use and the responsible teams are alerted. No file is exported, copied or uploaded by anybody. In a later phase, SAP's live PO / goods-receipt / return events (already received on `POST /api/sap/v1/events`) keep the open-PO and stock position current during the day.

Nothing in the decision rules changes. The engines keep reading the same datasets (F1…F4, Universe Master, BSR, masters); only **how the datasets get into the tool** changes.

---

1. Summary

	Today	**After phase 1 (direct pull)**	**After phase 3 (live position)**
Who loads the data	A planner, by hand, every morning	Nobody — the tool pulls it on a schedule (e.g. 04:30 after the BW night load)	Same, plus SAP events during the day
How	Excel / CSV / ZIP upload in Data Setup	**Linked server** (SQL query on the source database) or **API** (OData / REST) per dataset	+ `POST /events` applied to a live open-PO / stock position
Files	Exported, renamed and uploaded by people	**None**	None
Freshness	Whenever someone uploads (often mid-morning)	Previous day's close, ready before office hours; masters / plan as often as needed	Near real time for POs, receipts and returns
Completeness check	Visual	Source-side **count and quantity totals** for the same snapshot, master checks, held-aside rows	+ event order / cut-off checks
If a pull fails	Decisions run on yesterday's (or older) upload	Previous data kept, BW / IT / planning alerted, retried automatically	Same
SAP `bw_snapshot`	Upload time of F1	Snapshot time reported by the source	Same

Planned effort (tool side, after the open questions in §12 are answered): **phase 1 about 5–6 weeks**, **phase 2 (plan, GRC, business masters) about 2–3 weeks**, **phase 3 about 6–8 weeks**. Source-side work (views / services, users, network) is on top (§11).

---

2. How Data Setup works today

2.1 The datasets

The tool stores eleven datasets. Each upload creates a new **version** (gzip JSON in `dataset_versions`); the **manifest** says which version is current, and the engines read an in-memory snapshot of the current versions.

Key	Name on screen	What it is	Needed by	Upload
`f1`	F1 — Stock	Closing stock by Brand × MC × MRP × Flag × Colour (+ GM), company level	**Tool 1, SAP decisions**, Tool 2, Recommendation, Attribute Analysis	single
`f2`	F2 — Pending PO	Released open PO lines: PO no, GM, colour, delivery month, pending qty, value, vendor	**Tool 1, SAP, Tool 2** (its line list), Recommendation	single
`f3`	F3 — Sales Trend	Daily sales of the last 45 days by colour (`Day 1` = oldest)	**Tool 1, SAP**, Tool 2, Recommendation, Attribute Analysis	single
`f4`	F4 — Forecast	Monthly plan by Brand × MC × MRP × Flag (`MM.YYYY` columns)	**Tool 1, SAP**, Tool 2, Recommendation	single
`um`	Universe Master	GMs of the Single Concept MRP universe (routes a GM to engine B)	**Tool 1, SAP**, Tool 2, GRC	single
`bsr`	BSR — All Seasons	Per GM + colour: weeks on floor, average per-day sell-through %	**Tool 1, SAP** (engine A), Recommendation, GRC	**multi** (files accumulate)
`artmaster`	Article Master	GM → brand, division, MC, MRP, flag, season, style, fit, material, design, vendor	Resolve map (top priority), seasons, Recommendation A, Attribute Analysis	single
`colmaster`	Colour Master	Colour code → description	Tool 1 colour list, SAP colour matching	single
`attrrel`	Attr Relevance Master	Brand × MC × MRP × Flag → assessment dimension (Style / Fit / Material / Design)	Recommendation A	single
`catheadmap`	SCR Products / Cat Head Mapping	Brand × MC × MRP → buyer / category-head emails	**Creates logins**, Watch List approver routing, Tool 2 / GRC split	single
`grc`	GRC — Buying History	Goods received by Brand × GM × colour per month / week	GRC Back Testing only	own slot

Bold = mandatory: Tool 1 and SAP `/decisions` refuse to decide until all six of F1, F2, F3, F4, Universe Master and BSR are present (`Tool1Router.MissingRequired`).

2.2 The upload paths

Path (portal)	API	What it does
A file per slot	`IngestFile` (`Key`, `Append`)	Reads one dataset; BSR appends unless `Append=false`
📦 ZIP bundle	`IngestZip`	Recognises files by name (`F1_Stock…`, `UniverseMaster…`) or as raw exports; reports missing required files
🧩 Raw export import	`IngestRaw`	Recognises SAP/BW report layouts by their headers and converts them (stock: `Stk q + In Q` → `OP STK`; sales: `Calendar Day` / `Sales Qty` dates → `Day 1..N`; PO: `Sum of Pending Qty` → `Pending Qty`; article master; BSR)

Path (portal)	API	What it does
GRC slot	`IngestGrc`	Monthly (`Cal. year` row) or weekly (`WW.YYYY`) GRC layout

All paths end in the same commit: one **upload batch** (`UPL-yyyyMMdd-nnnn`), one new version per dataset, manifest update, upload log + audit line, cache invalidation; for `catheadmap` also the SCR login sync. History keeps every version, and any older version or whole upload can be made current again.

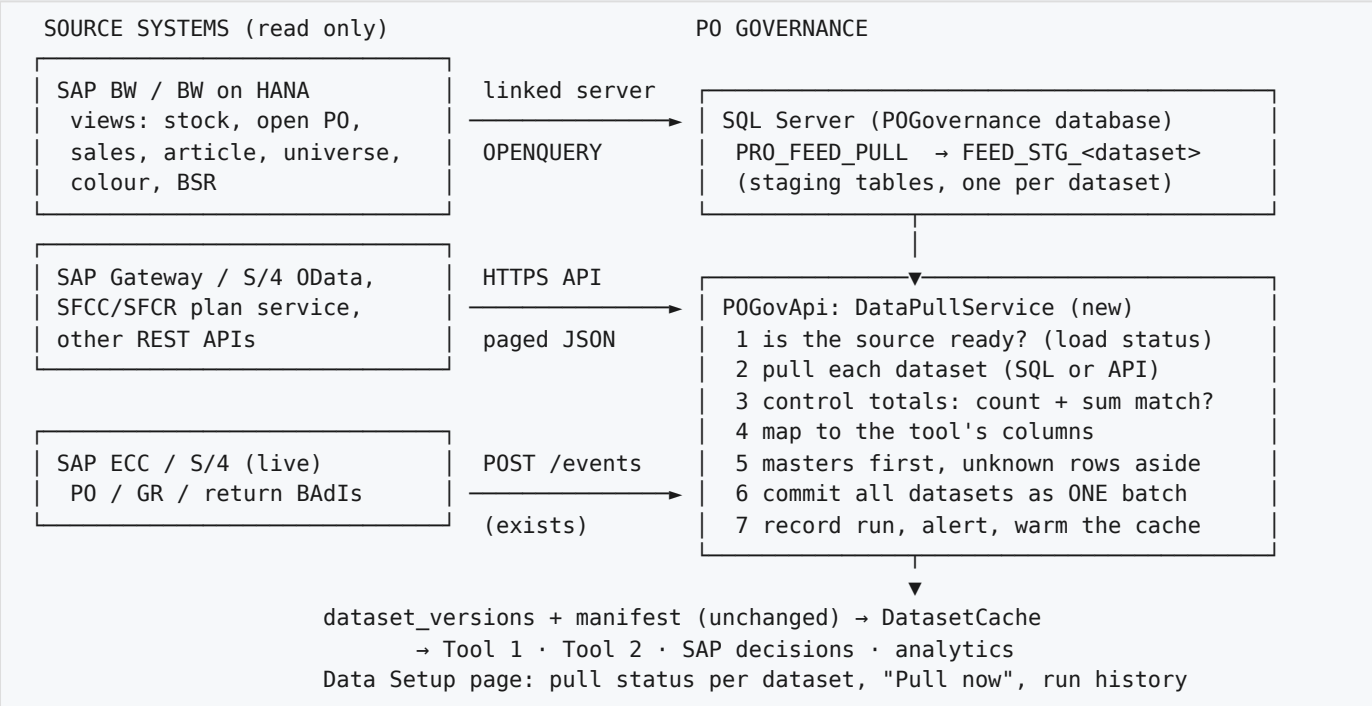
This commit is the seam for automation: the automatic pull produces the same rows and calls the same `SaveVersionAsync`, so the engines, History, restore and audit work unchanged.

2.3 Problems with the manual process

- The data is only as fresh as the last person who uploaded; a missed morning means decisions on old stock and POs.
- SAP `/decisions` already answers in real time, but on files that can be a day or more old (`bw_snapshot` = upload time of F1, not the real stock date).
- Large files are exported, renamed and uploaded by hand; mistakes (wrong file in a slot, old export) are only noticed by eye.
- BSR files accumulate per upload — an old season file can stay mixed in.
- No completeness check: a half-exported file loads as long as it has headers.

---

3. Target architecture



The principle: the source system owns the data and the business logic of the extract (which stock, which POs, cut-off); the tool only **reads** a defined view or service per dataset, at a time when the source says its data is complete, and checks that it received everything.

---

4. Connection options

4.1 Linked server (direct SQL read)

A **linked server** is a connection defined inside the tool's SQL Server to another database. A stored procedure in the POGovernance database runs the source query with `OPENQUERY`, so the filtering and

aggregation run **on the source** and only the result travels:

```
-- example: F1 stock of one snapshot, from a BW-on-HANA view exposed for the tool
INSERT INTO dbo.FEED_STG_F1 (Run_Id, Generic_Material, Colour, Brand_Type, Division, Mc_Cd, Mc_Desc,
Mrp, Flag, Stock_Qty)
SELECT @Run_Id, *
FROM OPENQUERY(BW_HANA, 'SELECT GENERIC_MATERIAL, COLOUR, BRAND_TYPE, DIVISION, MC_CD, MC_DESC, MRP,
FLAG, STOCK_QTY
                         FROM "POGOV"."V_STOCK" WHERE SNAPSHOT_DATE = CURRENT_DATE - 1');
```

Source	Linked-server provider
SAP BW on HANA / BW/4HANA, S/4HANA (HANA database)	SAP HANA ODBC driver (HDBODBC) via MSDASQL
A data warehouse on SQL Server	SQL Server (native)
Oracle / other DW	the vendor's OLE DB / ODBC provider

- The source team publishes **one read-only view per dataset** (`V_STOCK`, `V_OPEN_PO`, `V_SALES_45D`, …) with exactly the fields in §5; the tool never reads raw SAP tables.
- Login: a **read-only technical database user** on the source, mapped in the linked server (`sp_addlinkedsrvlogin`); no write rights.
- Best for **large datasets** (sales of 45 days, stock, open POs): set-based, fast, no paging.

4.2 API (OData / REST)

The tool calls an HTTP service of the source and receives JSON:

- **SAP BW queries published as OData** (BW query with the "OData" flag, SAP Gateway), **S/4HANA CDS views** exposed as OData services, the **SFCC / SFCR** plan service, or any REST API of another system.
- Paging with `$top` / `$skip` (or server-driven `__next` / skip tokens), `$select` to the needed fields, `$filter` on the snapshot date.
- Authentication with a **technical user** (Basic) or **OAuth 2.0 client credentials**.
- The URLs and tokens live in the tool's existing **API master per environment**: `API_BASE_URL` (base URL of each source per Development / Staging / Production) and `API_AUTHORIZATION` (credentials per API and environment) — the same mechanism the portal already uses, so test systems are used from staging and production systems from production automatically.
- Best for **masters and the plan**, for systems that do not allow database access, and where the source already has services.

4.3 Comparison and recommendation

	Linked server	API (OData / REST)
Volume (millions of rows)	✓ best	paging, slower
Needs DB access to the source	yes (driver, DB user, firewall to DB port)	no (HTTPS only)
Business logic location	source view	source service
Security review	DB-level read user	service user / OAuth
Change of a field	change the view + mapping	change the service + mapping
Typical use here	stock, open PO, sales, BSR, article	plan (SFCC/SFCR), colour master, SCR / business masters, sources without DB access

Recommendation: support both, configured per dataset, with one common pipeline (checks, mapping, commit). Start with **linked server to BW (on HANA)** for the heavy daily datasets and **OData** for the plan and any master that has a service. Which source and which connector per dataset is a configuration row (§8.1), not code — it can be switched later (e.g. from linked server to an OData service) without a new release.

Not chosen: SAP RFC / BAPI calls from the tool (needs the SAP .NET connector, loads the ERP during the day and duplicates BW logic — the live part is covered by the SAP events); file exports (what this design removes).

---

5. Dataset by dataset: source, connector, schedule and mapping

The **source fields** below follow the Developer Brief's data list; the **tool column** is what the code reads today (Appendix A). The source team provides a view / service with these fields; the pull maps them to the tool columns.

5.1 Overview

#	Tool dataset	Source object (to be provided)	Connector (proposed)	Frequency	Phase
1	`f1` Stock	BW closing stock, company level — view `V_STOCK`	linked server	daily after BW night load	1
2	`f2` Pending PO	BW released vendor POs, open qty — view `V_OPEN_PO`	linked server	daily	1
3	`f3` Sales Trend	BW POS sales of the last 45 days — view `V_SALES_45D`	linked server	daily	1
4	`um` Universe Master	BW — view `V_UNIVERSE`	linked server	daily	1
5	`artmaster` Article Master	BW — view `V_ARTICLE`	linked server	daily	1
6	`colmaster` Colour Master	SAP colour characteristic — view or OData	linked server / OData	daily	1
7	`bsr` BSR	BW best-seller report, all seasons — view `V_BSR`	linked server	weekly	1
8	`f4` Forecast / plan	SAP SFCC / SFCR plan	OData / REST	when the plan version changes (later hourly)	2
9	`grc` GRC buying history	BW goods receipts by month / week — view `V_GRC`	linked server	weekly	2
10	`catheadmap` SCR mapping	merchandising / HR system (buyer & category head per Brand × MC × MRP), if one exists	API or linked server	daily	2
11	`attrrel` Attr Relevance	planning-owned table; if no system holds it, maintained **in the tool** (edit screen, no Excel)	— / in-tool	when changed	2
—	Live PO changes, GR, returns	SAP R/3 events	`POST /events` (exists)	real time	3

Fixed rules from the Developer Brief that every source view follows: **company level** (all sites summed), **vendor POs only (no STO)**, open PO = **released** POs only, open qty = ordered − received + returned, **no real-time sales**.

5.2 Stock → `f1`

Source field	Tool column (`f1`)	Rule
Generic Material	`Generic Material`	leading zeros removed
Colour	`Color`	colour **code** (as in Colour Master)
Brand Type	`Brand Type`	

Source field	Tool column (`f1`)	Rule
Division	`DIV`	
MC CD, MC Desc	`MC CD`, `MC DESC`	
MRP	`MRP`	number
Flag	`BOGO/NON-BOGO`	`BOGO` / `NON-BOGO` exactly as in F4 / Universe
Stock Qty	`OP STK`	closing stock **incl. in-transit** (today's manual export adds `Stk q + In Q`) — **confirm**
Snapshot date / time	—	must match the snapshot of the run; becomes `bw_snapshot`

5.3 Open PO → `f2`

Source field	Tool column (`f2`)	Rule
PO Number (EKKO-EBELN)	`PO Number`	
Generic Material (MARA-SATNR), Colour, Brand Type, Division, MC CD, MC Desc, MRP, Flag	`Generic Material`, `Color`, `Brand Type`, `DIV`, `MC CD`, `MC DESC`, `MRP`, `BOGO/NON-BOGO`	
Delivery Date (first schedule line, EKET-EINDT)	`Delivery Month`	converted to **`MM.YYYY`**
Pending Qty	`Pending Qty`	ordered − received + returned, computed in the source view
PO Value	`PO Value`	needed for Tool 2 "₹ Value at Risk"
Vendor Name	`Vendor Name`	
Season, Article description	`Season`, `ART DESC`	resolve-map fallback when a GM is not in Article Master
PO item, ordered / received / returned qty, release date	—	kept for checks and phase 3

5.4 Sales → `f3`

The tool reads **one row per GM × colour** with `Day 1` … `Day 45` (oldest first). Either the source view returns that **wide** shape directly (preferred, far fewer rows) or it returns one row per day and the pull pivots it:

Source field	Tool column (`f3`)	Rule
Generic Material, Colour, Brand Type, MC CD, MRP, Flag	same as F1	
Date + Sales Qty (or Day 1…Day 45)	`Day 1` … `Day N`	N = `params.mixWindowDays` (45); `Day 1` = snapshot date − N + 1; missing days = 0

5.5 Universe Master → `um`

Source field	Tool column	Rule
Generic Material	`Generic Material`	presence = Single Concept
Brand Type, Division, MC CD, MC Desc, MRP, Flag	`Brand Type`, `Division`, `MC CD`, `MC DES`, `MRP`, `Flag (BOGO/NON-BOGO)`	
Concept, Valid From	—	only Single Concept rows valid on the snapshot date

5.6 Article Master → `artmaster`

Source field	Tool column	Rule
Generic Material, Article description	`Generic Material`, `ART DESC`	one row per GM
Brand Type, Division, MC CD, MC Desc, MRP, Flag, Season	`Brand Type`, `Division`, `MC CD`, `MC DESC`, `MRP`, `Flag (BOGO/NON-BOGO)`, `Season`	Season drives Current / Fallback season selection
Style, Fit, Product Material, Design	same names	Recommendation dimensions, Attribute Analysis
Vendor, Vendor Name	`Vendor`, `VENDOR NAME`	

5.7 Colour Master → `colmaster`

Source field	Tool column	Rule
Colour Code	`Colour`	the tool reads `Colour` / `Color` and `Colour Desc` / `Color Desc` / `Description`
Colour Description	`Colour Desc`	

5.8 BSR → `bsr` (weekly)

Source field	Tool column	Rule
Generic Material, Colour, MC CD, MRP, Flag	`Generic Material`, `Color`, `MC CD`, `MRP`, `Article Type`	
Weeks on Floor	`WEEK ON FLOOR`	
Avg Per-Day Sell-Thru %	`AVG PERDAY SELL THRU%`	percent number
Season, Snapshot Week	—	all seasons in one pull; a GM's season still comes from Article Master

Change of behaviour: today BSR **accumulates** over uploads; the pull **replaces** BSR with the complete weekly result. Between weekly pulls the current BSR stays.

5.9 Plan → `f4` (phase 2, API)

Source field	Tool column	Rule
Brand Type, MC CD, MRP, Flag	`Brand Type`, `MC CD`, `MRP`, `BOGO/NON-BOGO`	one row per bucket
Month + Plan Qty	`MM.YYYY` columns	pivot to wide
Plan Version	—	pulled only when the version changed

5.10 GRC → `grc` (phase 2)

Goods received per Brand × GM × colour × month (or week) with quantity and value, straight into the tool's GRC row shape (`brand, gm, artdesc, colour, period, periodType, qty, value`) — no Excel layout parsing any more.

5.11 Business-owned masters

- **SCR mapping (`catheadmap`)** and **Attr Relevance (`attrrel`)** are business decisions, not transactions. If a system holds them, they are pulled like any other dataset (SCR pulls keep creating logins). If not, they are **maintained in the tool** on a small edit screen (add / change / delete rows, history and audit) — so no Excel file is needed anywhere.
- **Parameters** and **Season Selection** stay in the Data Setup sidebar as today.

---

6. Source contract (to agree with the BW / SAP / IT teams)

Item	Rule
Objects	one read-only **view** (linked server) or **service entity set** (API) per dataset, with the fields of §5, field names fixed
Snapshot	every view / service filters on a **snapshot date**; all datasets of one run are read for the **same** snapshot
Readiness	a **load-status** view / endpoint per source: `Source, Snapshot_Date, Status (COMPLETE / RUNNING / FAILED), Completed_At, Cut_Off_Time` — e.g. from the BW process-chain log; the tool reads only when `COMPLETE`
Control totals	per dataset a **control query** for the same snapshot: `COUNT(*)` and `SUM(quantity)` (stock qty / pending qty / sales qty / plan qty) — the tool compares them with what it received
Codes	GM and article without leading zeros (or the tool strips them); colour = Colour Master code
Numbers / dates	numeric types (no text with separators); dates as date types; timestamps with time zone
Users	read-only technical DB user (linked server) / technical service user or OAuth client (API), one per environment (QA source for staging, production source for production)
Network	tool SQL Server → source DB port (linked server) and / or tool API server → source HTTPS (API); firewall rules per environment
Performance	views tuned for a full read of one snapshot within the pull window (target < 15 min for the nightly set)
Change control	a change of a view / service field is announced to the tool team (mapping update)

7. The pull process (DataPullService)

A new background service in POGovApi, following the pattern of the existing `BackupBackgroundService` (checks every few minutes, acts on its schedule, uses the business time zone `Asia/Kolkata`).

7.1 Schedule and settings

Setting	Default	Meaning
`DataPull:Enabled`	`false`	master switch (off until go-live)
`DataPull:DailyStartAt`	`04:30`	start checking the sources' load status
`DataPull:LateAfter`	`06:30`	sources not ready by then → "data late" alert
`DataPull:GiveUpAt`	`09:00`	stop waiting; decisions keep the previous data (alert sent)
`DataPull:PollMinutes`	`5`	readiness check interval
`DataPull:CommandTimeoutSeconds`	`900`	per linked-server query / API page
`DataPull:MaxRejectPct`, `MaxSwingPct`	`2`, `30`	held-aside rows limit; plausibility vs previous day
`DataPull:AlertTo`	BW, IT, planning	email recipients
per dataset (table `FEED_DATASET`)		connector, source object, frequency (daily / weekly / on version change), active

7.2 Steps of one run

1. **Ready?** Read the load status of every source involved. Proceed only when all report `COMPLETE` for a snapshot newer than the current one; else wait (step 1 again) until `GiveUpAt`.
2. **Pull.** For each due dataset (masters first): linked server → `PRO_FEED_PULL @Dataset` fills `FEED_STG_<dataset>` with `OPENQUERY`; API → page through the service and bulk-insert into the same staging table.

3. **Control totals.** Run the dataset's control query at the source; row count and quantity sum must equal what was staged. Any mismatch → the run **fails** (step 8), nothing is used.
4. **Map.** Staging rows → the tool's canonical columns (§5, Appendix A): `MM.YYYY` months, sales / plan pivot, leading zeros stripped, flags normalised.
5. **Reference checks.** Rows whose GM is in no master, or whose colour is not in Colour Master, are **held aside** (`FEED_REJECT`), not loaded; above `MaxRejectPct` the run fails.
6. **Plausibility.** Totals vs the current data within `MaxSwingPct` → otherwise a **warning** for planning (not a failure).
7. **Commit — one batch.** One batch `UPL-…` with source **`auto-pull`** and the snapshot as label; one new version per dataset through the same `SaveVersionAsync` the uploads use today; manifest updated once; BSR **replaced**; F4 only on a new plan version; `bw_snapshot` = the source's cut-off time. Cache invalidated; `DecisionWarmUpService` loads the new data within 30 s.
8. **On failure.** Nothing is committed, the **previous data stays current**, staging is kept for diagnosis, the run is recorded with the reason, an alert is emailed, and the Data Setup page shows the failed dataset in red. Retried at the next poll until `GiveUpAt`; an admin can press **Pull now** after the source is fixed.
9. **Record.** Run, per-dataset counts / totals / duration, held-aside rows; audit line "Data pull"; staging tables cleared after a successful commit.

7.3 Rules

- One snapshot is committed once; a re-run for the same snapshot (after a source correction) creates a new batch and becomes current (the older one stays in History).
- An older snapshot than the current one is never pulled automatically (an admin can force it).
- Two runs never overlap (the store lock used by uploads today).
- Intra-day datasets (e.g. plan on version change) commit on their own, as a separate batch, without touching the nightly datasets.

---

8. Changes in the tool

8.1 Database (new script `06_data_pull.sql`, schema v5)

Object	Purpose
`FEED_SOURCE` (Source_Key, Type `LINKED_SERVER` / `ODATA` / `REST`, Linked_Server_Name or Base_Url_Key, Status_Query / Status_Path, Active)	the source systems and how to reach them
`FEED_DATASET` (Dataset_Key, Source_Key, Source_Object (view / entity set), Control_Query / Control_Path, Frequency, Order, Active, Column_Map JSON)	what to pull per dataset — editable without a release
`FEED_STG_<dataset>` (F1, F2, F3, F4, UM, BSR, ARTICLE, COLOUR, GRC, SCR, ATTRREL)	staging per run
`FEED_RUN`, `FEED_RUN_DATASET` (rows, qty, control totals, duration, status, message)	run history
`FEED_REJECT` (Run_Id, Dataset_Key, Reason, Row_Json)	held-aside rows (kept 60 days)
`PRO_FEED_PULL`, `PRO_FEED_CONTROL`, `PRO_FEED_RUN` (`START` / `DATASET` / `FINISH` / `LIST` / `GET`)	QPS-style procedures (`@Cmd`, `@Error_Code`, `@Error_Message`)
Linked servers	created by the DBA per environment (`BW_HANA`, …) with a read-only login — not in the scripts

8.2 Code

Where	What
`Application/Services/V1/DataPull/`	`DataPullService` (steps 1–9), `PullMapper` (source fields → canonical rows, pivots), `PullValidator` (control totals, references, plausibility)
`Application/Interfaces/V1/DataPull/`	`IPullConnector` with two implementations: `LinkedServerConnector` (calls `PRO_FEED_PULL` / `PRO_FEED_CONTROL`) and `ODataConnector` / `RestConnector` (HttpClient, paging, API master URL + token per environment)
`Infrastructure/Services/V1/DataPullBackgroundService.cs`	schedule
`Infrastructure/Repositories/DataPullSqlServerRepository.cs`	the procedures
`Infrastructure/Services/V1/Mail/`	SMTP alert sender (the tool has no mail sender today)
`Application/Services/V1/Ingest/DataIngestService`	unchanged; the pull reuses `SaveVersionAsync` and the batch / audit / cache steps
`SapDecisionService.BwSnapshot`	read the snapshot time stored with F1 instead of the upload time
`POGovApi/Controllers/V1/DataPullController.cs`	`DataPullStatus`, `DataPullRuns`, `DataPullRunGet`, `DataPullNow`, `DataPullRejectsExport`, `FeedDatasetList` / `Save` (admin) + API_MASTER rows

8.3 Portal

- 📂 **Data Setup** becomes a **status page**: per dataset — source, connector, last pull (snapshot, rows, quantity total, status), next pull, held-aside rows (⬇ Excel); **Pull now** (admin); run history. The upload slots, ZIP and raw-import boxes are **removed**. Parameters and Season Selection stay.
- **Admin → Data sources**: the `FEED_SOURCE` / `FEED_DATASET` configuration (connector, view / service, frequency, active), **Test connection** and **Test pull (dry run)** per dataset.
- **SCR mapping / Attr Relevance** edit screens if no source system holds them (§5.11).
- 🗂 **History → Data uploads** lists the pulls (type "Automatic pull") — open, download as Excel and make an older snapshot current, as today.
- Header pill turns amber when today's data is late or a pull failed ("Data from 25 Oct").

8.4 Emergency fallback

With no uploads in normal operation, the fallback for a source outage is: keep using the previous snapshot (decisions continue; SAP answers can set `data_delayed`), or an admin makes an earlier snapshot current in History. Optionally an **admin-only** emergency upload can be kept behind a switch (`DataPull:AllowManualUpload`, default `false`) for the first months after go-live.

8.5 Fixes on the way (found while preparing this document)

- Upload size limit read from three different keys and hard-coded in the portal — becomes irrelevant once uploads are removed; keep one key if the emergency upload stays.
- Colour Master lookup does not accept `Colour Code` / `Colour Name` — the pull maps to the accepted names.
- BSR append vs replace — the pull always replaces.

---

9. Phase 3 — live open-PO and stock position from SAP events

SAP already sends PO, goods-receipt and return events (`POST /api/sap/v1/events`, stored in `SAP_EVENTS` with `Applied = 0`). Phase 3 applies them on top of the nightly snapshot so that decisions see POs raised, changed, received or returned **today**, and the same-day Tool 1 ledger can be switched off (handover §8 item 9).

```
live open PO (per PO item)   = pulled open PO (snapshot, cut-off T)
                               + PO_CREATED / PO_CHANGED / PO_RELEASED after T   (full current state
per PO)
                               − PO_CLOSED / PO_DELETED / RELEASE_REVOKED after T
                               − GR_POSTED  + GR_REVERSED                         (received)
                               + RETURN_POSTED − RETURN_REVERSED                  (only if a return
re-opens the qty — §12 Q1)
live stock adjustment        = + GR_POSTED − GR_REVERSED − RETURN_POSTED + RETURN_REVERSED   (after
T)
```

- Only events with `occurred_at` **after the cut-off T** reported by the source's load status are applied — no double counting (SAP spec v1.4 p.4).
- Each new nightly snapshot clears the previous day's adjustments and re-applies the events after its cut-off.
- PO events carry the full PO state; the newest per PO (by `seq`) wins.

Item	Change
Tables	`LIVE_OPEN_PO`, `LIVE_STOCK_ADJ`
Applier	background service applying `ACCEPTED` events in `seq` order every few seconds; `Applied = 1`
Engines	Tool 1 B / SAP decisions read open PO = live open PO; stock = F1 + live adjustment (switch `LivePosition:Enabled`)
Ledger	same-day Tool 1 ledger off when the live position is on

10. Testing, cut-over and rollback

1. **Connection tests** per environment: linked server (`SELECT` 1 row from each view), API (one page per service), load-status and control queries.
2. **Dry run** (`DataPull:DryRun = true`): pulls, checks and maps, writes run history and rejects, **commits nothing**.
3. **Parallel run, 2–3 weeks** on staging: the pull runs every night while production still uses today's uploads; a daily **reconciliation report** compares both per dataset (rows, totals) and the resulting Tool 2 decision mix and a sample of Tool 1 decisions.
4. **Go-live criteria**: 10 consecutive nights with sources ready before 06:30, control totals matching, held-aside rows < 0.5 %, decision differences explained.
5. **Go-live**: `DataPull:Enabled = true` in production; upload slots removed from Data Setup.
6. **Rollback**: switch the pull off and re-enable uploads (`AllowManualUpload`); History can make any earlier data current again.

Automated tests: mapping per dataset, pivots, control-total mismatch, held-aside rows, re-run of a snapshot, BSR replace, F4 version skip, failure keeps the previous data, connector paging (API mock), procedures (SQL Server suite with a local linked server loop-back).

11. Plan, effort and responsibilities

Phase	Scope	Tool team	Source / IT teams	Duration
0	Agree the source contract (§6), answer §12, network and users	2–3 days	views / services design, DB / service users, firewall	1–2 weeks
1	Pull for stock, open PO, sales, universe, article, colour, BSR; staging, control totals, run log, alerts, Data Setup status page, data-source admin, dry run	~4 weeks build + test	BW views on HANA (or DW), load-status view, linked-server setup by the DBA	5–6 weeks incl. parallel run
2	Plan via SFCC/SFCR API, GRC, SCR / Attr Relevance (pull or in-tool edit), remove uploads	~2–3 weeks	plan service, GRC view	2–3 weeks
3	Live open-PO / stock position from SAP events; ledger off	~5–6 weeks	confirm event rules (§12), bgRFC in production	6–8 weeks

Who	Responsible for
BW team	source views / OData queries, load status and cut-off, control totals
SAP team	plan service, events (built), GRT rule
DBA / IT	linked servers, HANA ODBC driver on the tool's SQL Server, DB users, firewall, SMTP relay
Planning	parallel-run review, Attr Relevance, parameters
Merchandising	SCR mapping source
Tool team	pull service, connectors, staging, checks, portal, alerts, tests, reconciliation report

12. Open questions (need answers before build)

#	Question	Owner	Why it matters
1	Does a goods return (122 / 161) **re-open** the pending PO quantity? (handover §8 item 6)	SAP + Planning	open-PO formula, phase 3
2	Where is the source data: **BW on HANA / BW/4HANA**, classic BW on another DB, or a separate data warehouse?	BW / IT	linked-server provider vs API
3	Is **database access** (linked server) from the tool's SQL Server to that source allowed, or only APIs?	IT security	connector per dataset
4	Which **load-status** object tells that the night load is complete, and what is the **cut-off time**? (handover §8 item 7)	BW	readiness, `bw_snapshot`, double counting
5	Does stock include **in-transit**?	BW + Planning	F1 = stock the engines use
6	Can the views provide PO Value, Vendor Name, Division, MC Desc, Season, Art Desc (open PO), MC Desc (stock), Brand Type / Division / MC Desc (universe), Art Desc (article)?	BW	fields used today (§5)
7	Sales view **wide** (`Day 1..45`) or one row per day?	BW	volume
8	BSR refresh day; all seasons in one result?	BW	BSR replace
9	Plan (SFCC/SFCR): service available? version field, Brand Type in the key, frequency	Planning + SAP	phase 2
10	Is there a system of record for the **SCR mapping** and **Attr Relevance**, or should the tool hold them?	Merchandising + Planning	§5.11
11	Who receives failure alerts; SMTP relay available?	IT + Planning	alerts
12	Approximate row counts (stock, open PO, sales 45 days)	BW	sizing, pull window
13	Keep an admin-only emergency upload for the first months after go-live?	Planning	§8.4

Appendix A — Columns the tool reads today, per dataset (exact headers)

Header matching for most engines ignores case and spaces (`SheetRow.Col`); the Recommendation, Attribute Analysis, GRC and Cat-Head lookups need the **exact** header — the pull therefore always writes the first name listed.

Dataset	Columns (first name = the one the pull writes; alternates accepted today)
f1	`Generic Material` · `Color` (Colour) · `Brand Type` · `DIV` (Division) · `MC CD` · `MC DESC` · `MRP` · `BOGO/NON-BOGO` (Flag, Article Type, Flag (BOGO/NON-BOGO)) · `OP STK` (Op Stk)
f2	`PO Number` (PO No, Purchase Order …) · `Generic Material` · `Color` · `Brand Type` · `DIV` · `MC CD` · `MC DESC` · `MRP` · `BOGO/NON-BOGO` · `Delivery Month` (`MM.YYYY`; DEL MONTH) · `Pending Qty` (Released Qty) · `PO Value` (PO Val, Value, PO Amount) · `Vendor Name` (Vendor) · `Season` · `ART DESC`
f3	`Generic Material` · `Color` · `Brand Type` · `MC CD` · `MRP` · `BOGO/NON-BOGO` · `Day 1` … `Day N` (`D N`; oldest first) — or `MM.YYYY` month columns / `CY APR & MAY AVG`
f4	`Brand Type` · `MC CD` · `MRP` · `BOGO/NON-BOGO` · `MM.YYYY` month columns
um	`Generic Material` · `Brand Type` · `Division` · `MC CD` · `MC DES` · `MRP` · `Flag (BOGO/NON-BOGO)`
bsr	`Generic Material` · `Color` · `MC CD` · `MRP` · `Article Type` (BOGO/NON-BOGO, Flag) · `WEEK ON FLOOR` (WOF) · `AVG PERDAY SELL THRU%`
artmaster	`Generic Material` · `ART DESC` · `Brand Type` · `Division` · `MC CD` · `MC DESC` · `MRP` · `Flag (BOGO/NON-BOGO)` · `Season` · `Style` · `Fit` · `Product Material` · `Design` · `Vendor` · `VENDOR NAME` (any other column = an attribute in Attribute Analysis)
colmaster	`Colour` (Color) · `Colour Desc` (Color Desc, Description, Desc)
attrrel	`Brand Type` · `MC CD` · `MRP` · `BOGO/NON-BOGO` · `Assessment Dimension`
catheadmap	`Brand Type` · `MC CD` · `MRP` · buyer / category-head email columns (Level 1/2 → buyer, Level 3 → category head; `Category_Email_ID` for Tool 2 / GRC)
grc	own layout: `Brand Type`, `Generic Material`, art description, `Colour`; monthly `MM.YYYY` under a `Cal. year` row or weekly `WW.YYYY` with `GRC Q` / `GRC V`

Formats: delivery / plan months `MM.YYYY`; day columns `Day N`; numbers as plain numbers; per-day sell-through as a percentage.

Appendix B — Sources

- `legacy/python-app/docs/PO_Governance_Developer_Brief_Stage1_2_3.pdf` §04 (data sources and frequencies, server steps), `PO_Governance_SAP_API_Specification_v1.4.pdf` (cut-off rule, SAP fields), `DEVELOPER-HANDOVER.md` §5 and §8 (future sources, open items 6, 7, 9), `PO_Governance_Functional_Logic_Guide.pdf` p.2 (datasets).
- Code: `DecisionEngine/Model/DatasetKeys.cs`, `Application/Services/V1/Ingest/LegacyImport.cs`, `DataIngestService.cs`, `Application/Services/V1/Platform/DataService.cs`, `Infrastructure/Services/V1/DatasetCache.cs`, `BackupService.cs`, `database/sqlserver/04_api_master.sql` (`API_BASE_URL`, `API_AUTHORIZATION`), `05_sap_interface.sql` (`SAP_EVENTS`).